

3. Bubble Sort with Big-O Analysis

Aim

To sort the elements of an array using the **Bubble Sort algorithm** and analyze its Big-O time complexity.

Algorithm

1. Read an array of elements.
2. Compare adjacent elements.
3. Swap them if they are in the wrong order.
4. Repeat the process for all elements.
5. Continue until the array is sorted.
6. Display the sorted array.

Procedure

1. Start the program.
2. Read the number of elements.
3. Read the array elements.
4. Compare adjacent elements using nested loops.
5. Swap elements when the first element is greater than the second.
6. Repeat until all elements are sorted.
7. Display the sorted array.



The screenshot shows a Python IDE with a file named 'main.py'. The code implements the Bubble Sort algorithm. It prompts the user to enter the number of elements (5) and then the elements themselves (1 4 8 6 3). The output shows the sorted array: [1, 3, 4, 6, 8].

```
main.py Visualize Run ▶ Output
1 n = int(input("Enter number of elements: "))
2 a = list(map(int, input("Enter elements: ").split()))
3
4 for i in range(n - 1):
5     for j in range(n - i - 1):
6         if a[j] > a[j + 1]:
7             a[j], a[j + 1] = a[j + 1], a[j]
8
9 print("Sorted Array:", a)
```

Enter number of elements: 5
Enter elements: 1 4 8 6 3
Sorted Array: [1, 3, 4, 6, 8]

Result

Thus, the given array was successfully sorted using Bubble Sort, and its time complexity was determined as **$O(n^2)$** .